ELI — Adaptive Inference Governor

ELI v2.5.21

October 2026

Contents

Adaptive Inference Governor — Status and Remaining Plan **1**
0. The one-sentence problem . . . 1
1. What ELI already adapts to (do not rebuild) . . . 1
2. The gaps and where each stands . . . 2
3. Design for what remains: the Adaptive Inference Governor . . . 3
4. Cross-machine behaviour to aim for . . . 4
5. Phases . . . 5
6. Verification . . . 5
7. Risks and non-goals . . . 5
8. Mixture-of-experts models on a small GPU . . . 5
9. The startup dialog's own model loader . . . 5

Adaptive Inference Governor — Status and Remaining Plan

> **Updated for v2.4.84.** Most of the gaps this plan identified in June are closed; the two that need a governor (throughput-aware think decision and per-call wall-time projection) are still open. Status per gap below; no governor code exists yet.

Scope: make ELI's reasoning and inference *policy* (think or no-think, token budget, pass count, recovery) a pure function of the runtime environment and the task, so one redistributed binary behaves correctly on any machine with any model and no per-machine tuning.

Origin: a 2026-06-17 session in which `EXPLAIN_COGNITION_RUNTIME` burned a 273 s empty compact-synthesis pass, then two 539 s empty standard-synthesis passes, before a two-stage chain-of-thought fallback (470 s and 502 s): about 38 minutes for one question. The working final answer was 4,071 characters (about 1,018 tokens) in 502 s, roughly 2 tokens per second.

---

0. The one-sentence problem

ELI adapts its inference to **memory** (VRAM, context length) and **model size**, but not to **measured compute speed** (tokens per second). On a slow machine the same prompt that works on the development box can produce multi-minute turns.

---

1. What ELI already adapts to (do not rebuild)

Capability	Where	What it adapts to
Model-agnostic resolution (no baked model)	`gguf_inference.get_model_path`	settings, environment
Chat family and template	`cognition/chat_template.py`, `model_identity.py`, `model_output_tokens.py`	the model's own Jinja template (format, reasoning and end-of-turn markers), not the file name
Thinking-channel detection	`gguf_inference._is_thinking_model` → `model_identity.is_thinking_model`	the embedded chat template, then architecture; no name list
Trained context length	`core/startup_hardware_optimizer.read_model_max_ctx`	`ELI_MODEL_TRAIN_CTX`, then real GGUF `context_length`, else 8192; no filename table
Load plan, measured before the load	`core/load_plan.py`	llama.cpp's load simulation on every GPU, each card's free memory and measured headroom, fit priority; the RAM budget on the processor
Estimate where llama.cpp cannot simulate	`hardware_profile.py` (`smart_fit_config`, `_kv_cache_mb`, `_compute_graph_reserve_mb`)	context, layers, batch
Load retry	`gguf_inference._build_with_plan`	a failed load is planned again for the memory free then, then on the processor
Per-mode token budget	`cognition/reasoning_modes.py`	`n_ctx` (20 % of the window for quick, 30 % otherwise), prompt pressure, complexity, mode
Size tier → budget scale	`core/model_tier.py` `detect_tier`, `tier_scale`	model size
Live decode-speed EMA	`model_tier.record_speed`, `measured_tok_s` (fed from the non-stream generation path in `gguf_inference`)	measured tokens per second
Speed-capped pass count	`model_tier.speed_passes`	measured tokens per second (thresholds `ELI_SLOW_TPS`=5, `ELI_FAST_TPS`=15)
No-think on utility calls	`gguf_inference._no_think_prefill`, `force_no_think()`	structured output, small budgets (< 1024), quick mode
Redistribution-aware settings	`core/runtime_settings.py`	machine moves

The speed EMA is consumed in exactly one place, `speed_passes()`. Nothing in the think decision, the token budget or mode selection reads it. `speed_passes()` reduces how many full generations a mode runs and never caps output length.

---

2. The gaps and where each stands

Gap	Was	Status
A thinking detection was a filename allowlist	`_is_thinking_model` matched `qwen3`, `deepseek-r1`, …	**Closed.** Detection reads the chat template, then the architecture.
B the think decision and token budget ignored throughput	no tok/s term in `_no_think_prefill` or the budget	**Open.** The inputs are `structured`, `max_tokens < 1024`, the quick mode, the `force_no_think()` scope and `ELI_MODEL_THINK`.
C no wall-time projection	nothing computes "at the live EMA this call takes about N seconds"	**Open.**
D empty output was not self-correcting	an unterminated `<think>` returned empty, then a full-price retry	**Partly closed.** An empty non-stream response is retried once with `force_no_think` in the engine (`_get_chat_response`), and an empty stream is retried once with no-think in `gguf_inference`. There is still no reserved answer budget.
E context sizing undershot the trained context	a filename table returned 32768 for `qwen3`	**Closed.** GGUF metadata is authoritative.
F `force_no_think` was a point patch	one call site	**Partly closed.** It now wraps compact grounded synthesis, the chain-of-thought stage prose, the correction repair, and the empty-response and stream retries in `_get_chat_response`. The standard synthesis fallback (`_synthesize_answer`) and coding-agent fix generation for `SELF_IMPROVE` are not wrapped.
G redundant proactive churn	`persona_updater` rebuilt the same overlay every tick	**Closed.** Persona and user-profile overlays are debounced (120 s) with a content dirty-check.

Note on gap G: `persona_updater` is deterministic (database reads and regexes); the cost was redundant CPU and SQLite work, not inference-lock contention.

---

3. Design for what remains: the Adaptive Inference Governor

A single policy object that every model call routes its *decision* through. It consumes `reasoning_modes`, `model_tier` and `hardware_profile`; it does not replace them.

3.1 Inputs (all available at runtime)

- **Model capability**: `model_identity.is_thinking_model` (template, architecture), cached per model.
- **Throughput**: `model_tier.measured_tok_s()`, with a cold-start default from `detect_tier()` until the first measurement.
- **Context**: `n_ctx` from the runtime snapshot and `n_ctx_train` from metadata.
- **Task class**: below.

3.2 Task classes

1. **STRUCTURED**: JSON, routing, plan graphs. Never thinks (already true via `structured=True`).
2. **GROUNDED_SYNTHESIS**: evidence already gathered; the call only phrases it (the `EXPLAIN_*` actions, RAG answer synthesis, fix-patch rendering). Thinking adds nothing; force no-think.
3. **OPEN_REASONING**: genuine multi-step reasoning (chain of thought, tree of thoughts, constitutional, self-consistency, open chat on a hard question). Thinking is allowed but bounded by throughput.

3.3 The decision (a pure function)

```
policy(task_class, model_caps, tok_s, n_ctx, prompt_tokens) -> {
    think: bool,
    think_budget: int,     # max tokens allowed inside <think>
    answer_budget: int,    # reserved tokens after </think>
    passes: int,           # via speed_passes()
}
```

- A model that does not think: `think=False` always.
- STRUCTURED or GROUNDED_SYNTHESIS: `think=False`.
- OPEN_REASONING: at or above `ELI_FAST_TPS` think with the full budget; between slow and fast scale `think_budget` so the projected wall time is within the target; at or below `ELI_SLOW_TPS` turn thinking off, or hard-cap `think_budget`.
- Always reserve `answer_budget`, so a thinking call cannot exhaust its budget before it answers.

3.4 Integration points

- `gguf_inference._no_think_prefill`: consult the governor instead of only `structured`, `< 1024` and the environment; keep `force_no_think()` as the primitive and `ELI_MODEL_THINK` as the hard user override.
- `reasoning_modes`: split the target tokens into `think_budget` and `answer_budget`.
- Engine synthesis sites: tag `_compact_grounded_synthesis` and `_synthesize_answer` as GROUNDED_SYNTHESIS and the reasoning-mode runners as OPEN_REASONING.
- Feed the speed EMA from the stream path as well, so quick-mode chat updates it.

3.5 Configuration

`ELI_SLOW_TPS` and `ELI_FAST_TPS` exist (5 and 15). A soft per-turn wall-time target (`ELI_TURN_LATENCY_TARGET_S`) is proposed and does not exist yet. `ELI_MODEL_THINK` is the hard override. Nothing machine-specific is persisted.

---

4. Cross-machine behaviour to aim for

Machine and model	tok/s	grounded actions (`EXPLAIN_*`)	hard chat
8 GB GPU with a 30B MoE	about 2	no-think, one pass	bounded think, within the target, never empty
24 GB GPU with a 7B	about 40	no-think	full think, full budget
CPU-only with a 7B	about 3	no-think	think off or hard-capped, with an answer reserve
any machine with a non-thinking model	n/a	normal answer	normal answer

Machine and model	tok/s	grounded actions (EXPLAIN_*)	hard chat
any machine with an unfamiliar reasoning model	any	detected through the template	handled

5. Phases

1. **Governor core** for grounded synthesis: the policy function, no-think wired through `_no_think_prefill`, coding-agent fix rendering tagged.
2. **Throughput-bounded open reasoning** with an answer reserve: split budgets in `reasoning_modes`.
3. **EMA from streaming** so quick mode updates the measured speed.

Each phase is independently shippable.

6. Verification

- Unit: `policy(...)` truth table over task class × throughput tier × model-thinks.
- No-model engine tests in the style of `tests/test_*_no_gguf*.py`.
- Behavioural: `EXPLAIN_MEMORY_RUNTIME` and `EXPLAIN_COGNITION_RUNTIME` synthesis is non-empty on the first pass and runs no-think.
- Speed simulation: inject a fake `measured_tok_s()` and assert the think flag and budget flip.
- Existing guards that must stay green: `tests/test_gguf_think_stop_collision.py`, `tests/test_reasoning_mode_co` `tests/test_cot_think_budget.py`.

7. Risks and non-goals

- **Over-suppressing thinking** on hard turns at mid speed: only GROUNDED_SYNTHESIS is unconditionally no-think; OPEN_REASONING keeps thinking with a budget above `slow`.
- **Non-goals:** changing model selection, the agent set, persona content or the grounding contract; removing `ELI_MODEL_THINK`.

8. Mixture-of-experts models on a small GPU

`eli/core/moe_offload.py`. A mixture-of-experts model uses a few experts per token, and the experts are most of the file. Splitting such a model by layers puts mostly idle experts on the GPU. With the offload on, every layer's attention and shared weights go to the GPU and the expert tensors stay in system memory (llama.cpp's tensor buffer override, applied to the model parameters while the model is built, because llama-cpp-python does not expose it).

- Measured on an RTX 2060 SUPER (8 GB), Qwen3.6-35B-A3B Q4_K_M, 1,500-token prompt: 8 GPU layers gave 71 tok/s prompt and 8.3 tok/s generation; all layers with experts in RAM gave 125 tok/s and 21.8 tok/s. Load time fell from 63 s to 5 s. Through `load_model` the same model went from 10.7 to 20.8 tok/s.
- When: the model is MoE (`GGUFModelProfile.is_moe`), it would not fit in free VRAM, and available RAM covers about three quarters of the expert weights. Otherwise the ordinary layer split is used.
- Switch: the "Expert offload (MoE)" choice in the startup dialog, the setting `moe_expert_offload` or env `ELI_MOE_EXPERT_OFFLOAD = auto` (default), `on`, `off`. `ELI_MOE_EXPERT_FRACTION` adjusts the assumed expert share of the file (default 0.90).
- The live runtime snapshot carries `moe_expert_offload`.

9. The startup dialog's own model loader

`eli/gui/eli_pro_audio_gui_v2_0.py` builds `Llama(...)` itself for the "Load selected model now" path — it does not go through `gguf_inference.load_model()`. The MoE expert-offload hook (section 8) is wired

into that loader too now; before this it was saved by the toggle but never reached the actual load, so the setting did nothing when a model was loaded from the startup dialog.

Measured against a live session (Qwen3.6-35B-A3B, ctx=13000, batch=512, RTX 2060 SUPER): 7 GPU layers without offload gave 6.5 tok/s; the offload's 41 layers (one past the model's 40 blocks — llama.cpp's own "all layers" convention, not an error) with experts in RAM gave 13.3 tok/s. Real, but the console printed the pre-offload layer count ("GPU-layer load parameter: 7") before a later line showed the actual loaded count ("gpu_layers=41") with no explanation connecting them — fixed, the first line now says when and by how much MoE will raise it. The bigger cost per chat turn is architectural, not this: chain-of-thought runs two full model calls (a private scratchpad, then the final answer) against 6,000–7,700-token prompts each, and MoE's prompt-processing gain (~1.75x) is smaller than its generation gain (~2x) — doubling generation speed does not halve a turn dominated by two large, mostly-prompt-bound calls. The second call no longer reads its prompt again: every pass of a turn is given the same system prompt (the engine builds it once per turn and keeps a quarter of the room for each pass's message), and the model's state at its end is kept between the passes (`gguf_inference._resume_from_saved_state`, see `inference_and_hardware.md`). Measured on this model: the second pass 63.8 s before, 3.2 s after. The first pass still reads the whole prompt (about 65 s for 7.5k tokens on this card); shrinking that prompt is separate work.

The footnote explaining the bump wasn't enough — the "HW Profile (recommended)" number itself, in `hardware_profile.recommend()`, still showed the pre-offload layer count, and that number is also what gets saved as the canonical pin (`hw_profile_n_gpu_layers`) other load paths read back. MoE is now decided before the recommendation is built, so `rec.n_gpu_layers` — the number this panel shows and the number that gets pinned — is the one that actually loads, with one reasoning line explaining why, not a separate figure with no visible link to it. This exposed a second bug: the VRAM-headroom check that sizes batch was reading "all layers on GPU" as "the full model is in VRAM," which is false under MoE (most of it is in RAM) — it would have computed a large phantom deficit and needlessly crushed the batch size. It now sizes off the plan's actual resident-on-GPU figure when MoE applies.